LaTeX Generation for Mathematical Expressions: An experiment with different models

Group 32

Department of ICT
University of Science and Technology of Hanoi

March 02, 2026

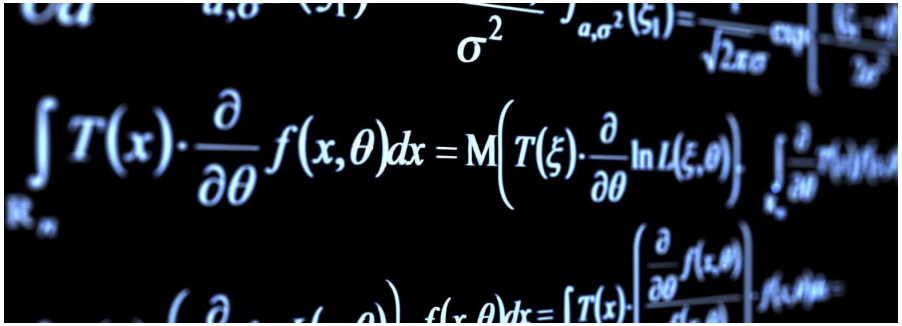
Table of Contents

1. Introduction
2. Dataset
3. Methodology
4. Metrics
5. Results
6. Discussion & Future Work

Outline

1. Introduction
2. Dataset
3. Methodology
4. Metrics
5. Results
6. Discussion & Future Work

Introduction



Motivation

- The need to digitize or modify documents that contain mathematical expressions: mathematical content exists in non-editable formats.
- Unlike plain text: Depend heavily on spatial layout and hierarchical structure.
- Writing equations directly in LaTeX can be challenging.

Introduction

$\sum$	\sum	$\bigcap$	\bigcap	$\bigodot$	\bigodot
$\prod$	\prod	$\bigcup$	\bigcup	$\bigotimes$	\bigotimes
$\coprod$	\coprod	$\bigsqcup$	\bigsqcup	$\bigoplus$	\bigoplus
$\int$	\int	$\bigvee$	\bigvee	$\biguplus$	\biguplus
$\oint$	\oint	$\bigwedge$	\bigwedge		

Table 7: Variable-sized Symbols

\arccos	\cos	\csc	\exp	\ker	\limsup	\min	\sinh
\arcsin	\cosh	\deg	\gcd	\lg	\ln	\Pr	\sup
\arctan	\cot	\det	\hom	\lim	\log	\sec	\tan
\arg	\coth	\dim	\inf	\liminf	\max	\sin	\tanh

Table 8: Log-like Symbols

Objective

- Deploy and compare three models that generate LaTeX code from images of mathematical expressions: CNN-LSTM, Vision Transformer (ViT), and hybrid Convolutional Transformer.
- Deploy a web application that converts uploaded images into LaTeX.
- Since it is an active and hard research topic in machine learning and AI due to its interdisciplinary nature, the project focuses on experiments and study new ideas.

Outline

1. Introduction
2. Dataset
3. Methodology
4. Metrics
5. Results
6. Discussion & Future Work

Dataset

formula string · *lengths* 1 481	filename string · *lengths* 14 14	image image · *width (px)* 320 320
\widetilde \gamma _ { \mathrm { h o p f } } \simeq \sum _ { n > 0 }…	66667cee5b.png	$_{\mathrm{opf}} \simeq \sum_{n>0} \widetilde{G}_n \frac{(-a)^n}{2^{2n-1}}$
({ \cal L } _ { a } g) _ { i j } = 0 , \ \ \ \ ({ \cal L } _ { a …	1cbb05a562.png	$_a g)_{ij} = 0, \quad (\mathcal{L}_a H)_{ijk} = 0,$
S _ { s t a t } = 2 \pi \sqrt { N _ { 5 } ^ { (1) } N _ { 5 } ^ {…	ed164cc822.png	$_{tat} = 2\pi\sqrt{N_5^{(1)} N_5^{(2)} N_5^{(3)}} \left(\sqrt{n} + \sqrt{\bar{n}}\right)$

Figure: Sample entries from the Dataset

Dataset: im2latex-100k

- Consists of approximately 100,000 images of mathematical expressions with their corresponding file names and LaTeX formula.
- Splitting: 70% training, 15% validation and 15% testing.

Outline

1. Introduction
2. Dataset
3. Methodology
4. Metrics
5. Results
6. Discussion & Future Work

LaTeX Vocabulary Building

Pipeline

1. Tokenize training LaTeX sequences by finding all characters, or strings matching categories:
   - LaTeX commands (Eg: \frac, \sum, \int)
   - Common LaTeX symbols: {, }, _, $, %, &, #, ^
   - A single digit (Eg: 0, 1, 2)
   - A single letter (Eg: x, y, a)
   - Any non-whitespace character (Eg: =, +, -)
   - Four special symbols: <sos>, <eos>, <pad>, <unk>.
2. Build LaTex Vocabulary with the token list, together with four special tokens.

Data Preprocessing

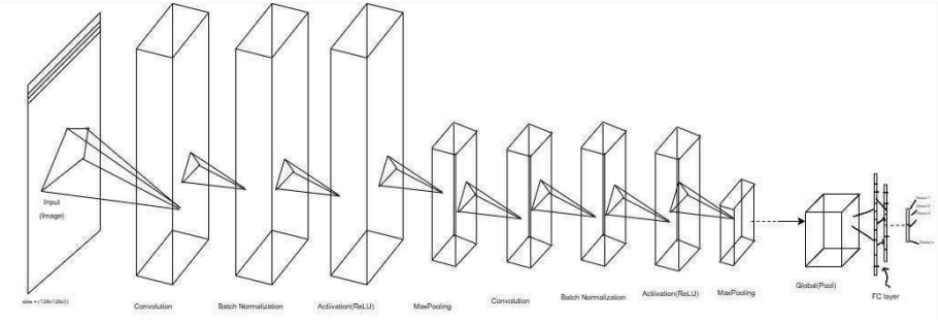
Image Preprocessing

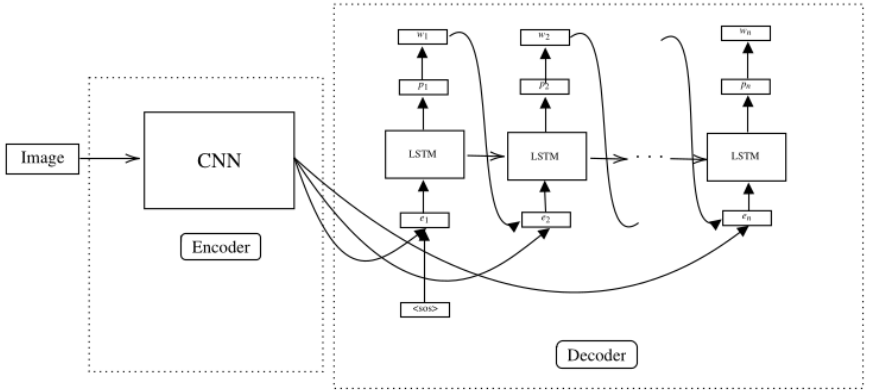
- Convert the images to grayscale.
- Resize the images to 50×200.
- Normalize the pixel values to $[0, 1]$.
- Convert the images to tensors.

LaTeX Sequence Preprocessing

- Add <sos> at the start and <eos> at the end.
- Tokenize each sequence by matching with predefined token categories.
- Convert each tokenized sequence into a list of token indices based on the vocabulary.
- Convert index lists into tensors.
- Pad tensors with <pad> index: all sequences have the same length.

Model Architecture: CNN - LSTM

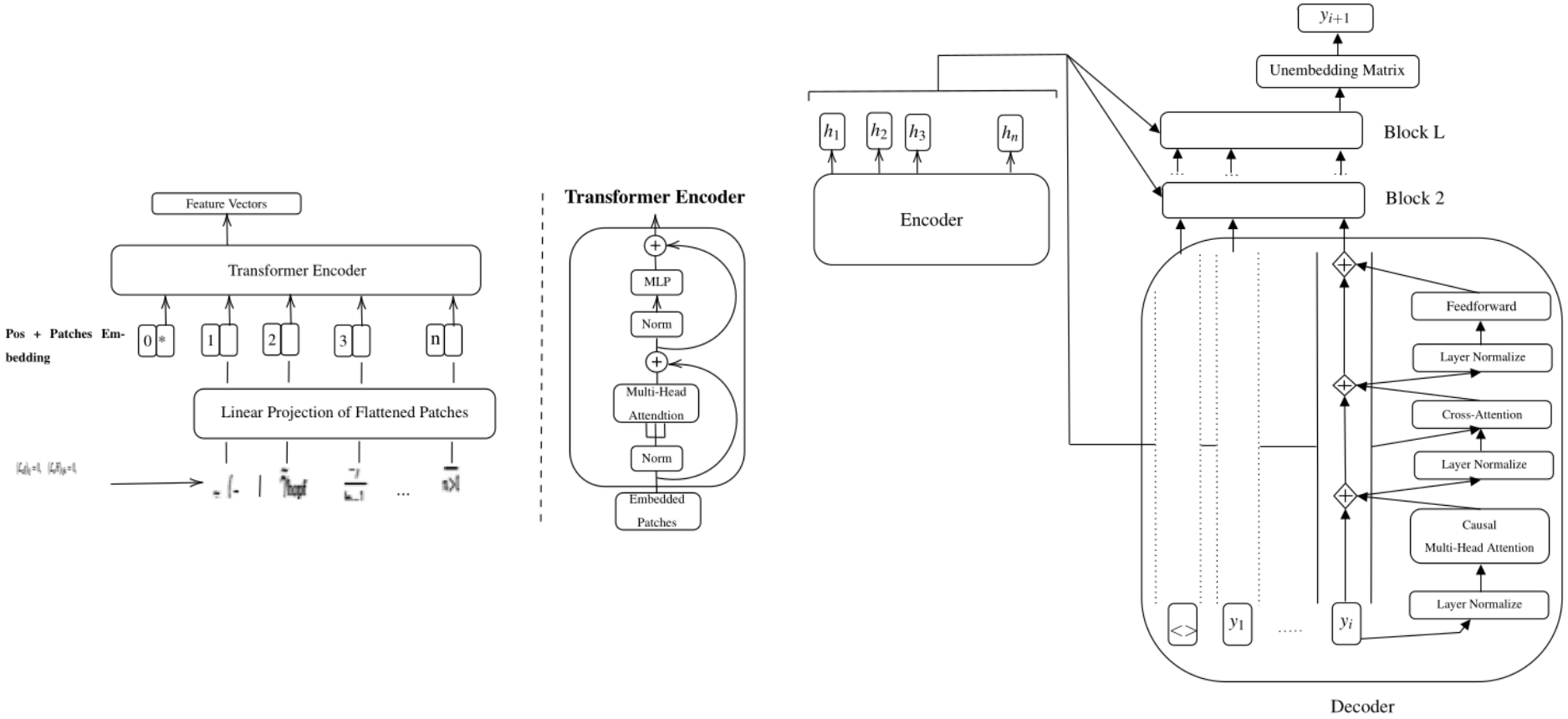




Key Features

- **CNN Encoder**: Series of convolutional blocks, focuses on local features.
- **LSTM Decoder**: Generates tokens sequentially based on the CNN context and previously predicted tokens.

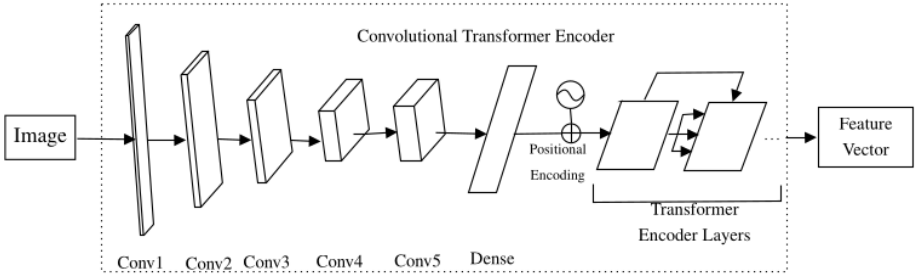
Model Architecture: ViT - Transformer

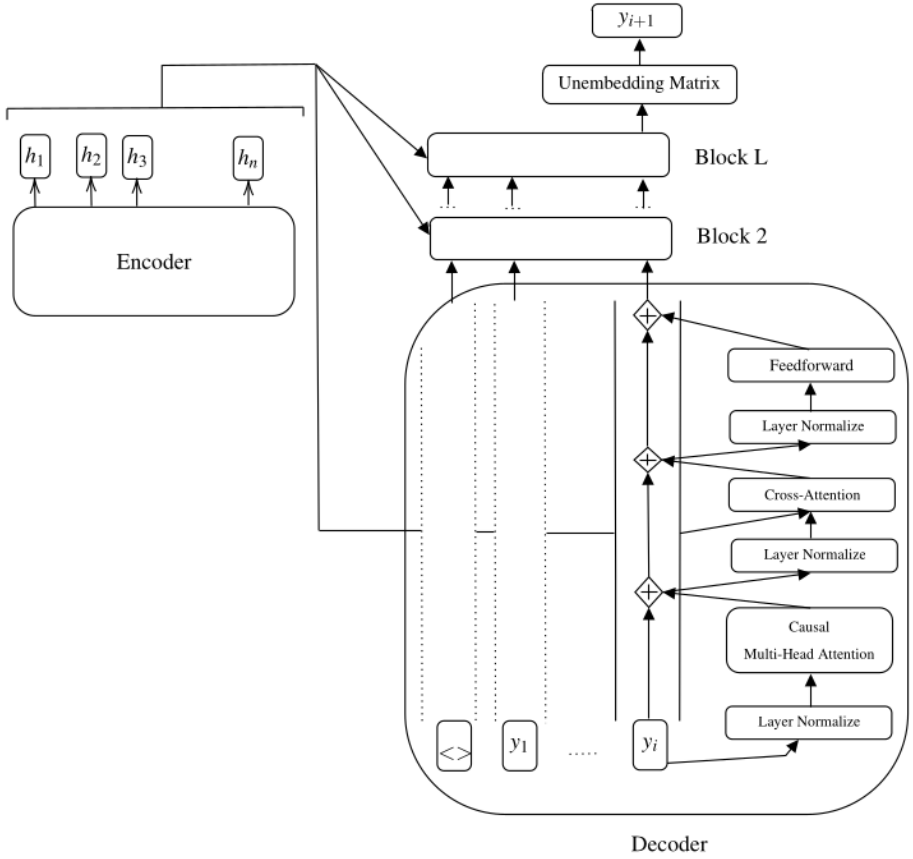


Key Features

- Both have Attention mechanism.
- **ViT Encoder**: Divide the image into patches, position embedding.
- **Transformer Decoder**: Generates the output sequence step-by-step.

Model Architecture: Convolutional Transformer





Key Features

- Both have Attention mechanism.
- **Convolutional Transformer Encoder**: Combining local feature extraction with the global attention.

Decode: Beam Search

Beam Search

- **What:** A decoding algorithm used in sequence generation models, keeps the top k highest probability sequences at each step.
- **Why:** Helps find a better overall sequence than greedy search.

Algorithm

1. **Initiation:** Start with an <sos> (initial beam).
2. **Generation:** Model generate all possible next tokens with their probabilities.
3. **Top k Selection:** Keep the top k highest-scoring sequences (beams).
4. **Iteration & Expansion:** Extend each beam with possible tokens.
5. **Re-evaluation & Pruning:** Score all candidates and keep top k.
6. **Termination:** Repeat steps 4–5 until <eos> is generated or the maximum sequence length is reached.

Web Application Deployment

Technology Stack

- **Backend:** Flask (Python)
- **Frontend:** HTML, CSS, JS
- **Inference:** PyTorch
- **Rendering:** MathJax

Key Features

- Drag & drop image upload
- Compare **3 models** at once
- LaTeX code + rendered output
- One-click copy to clipboard

Web Application Deployment

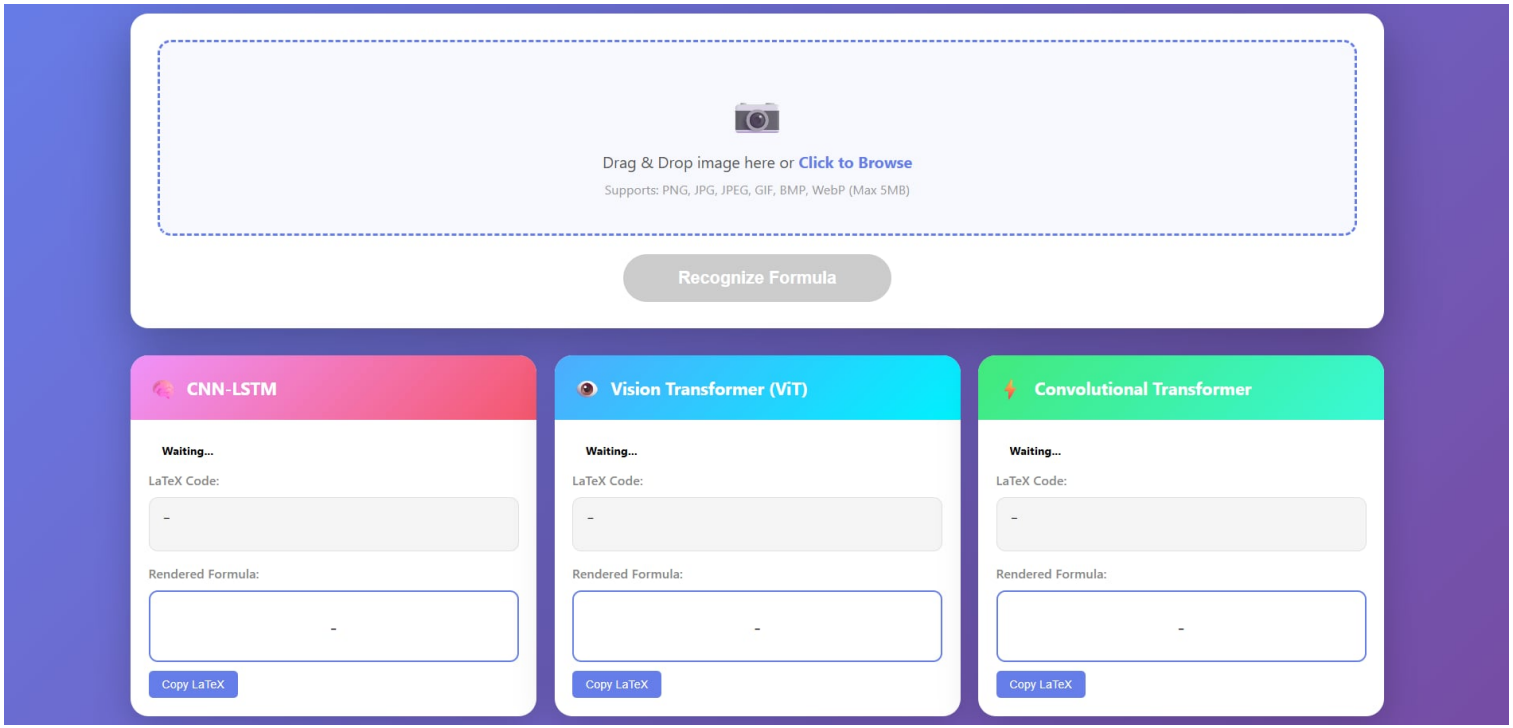


Figure: Web application interface for Generating LaTeX code for Mathematical Expression.

Processing Pipeline

Upload image $\rightarrow$ Preprocess (Grayscale, Resize 50×200) $\rightarrow$ Inference (3 models + Beam Search) $\rightarrow$ Token-to-LaTeX $\rightarrow$ MathJax render

Outline

1. Introduction
2. Dataset
3. Methodology
4. Metrics
5. Results
6. Discussion & Future Work

Metrics: Levenshtein Distance

Levenshtein Distance

- Measures the minimum number of single-character edits (insertion, deletion, substitution) required to transform one string into another, then normalizes the distance to account for differences in sequence length.
- The score ranges from 0 to 1, where 1 indicates no similarity and 0 indicates that the two sequences are completely identical.

		s	e	t	t	i	n	g
	0	1	2	3	4	5	6	7
s	1	0	1	2	3	4	5	6
i	2	1	1	2	3	3	4	5
t	3	2	2	1	2	3	4	5
t	4	3	3	2	1	2	3	4
m	5	4	4	3	2	2	3	4
g	6	5	5	4	3	3	3	3

Figure: Example of Levenshtein distance operations

Metrics: BLEU Score

BLEU Score

- Measures the overlap of n-grams between generated text and reference translations, using the geometric mean to combine n-gram precision scores for phrase-level accuracy and applying a brevity penalty (BP) to penalize generated text shorter than the ground truth.
- The BLEU score ranges from 0 to 1, where higher values indicate better agreement between the predicted output and the reference text in terms of both content and structure.

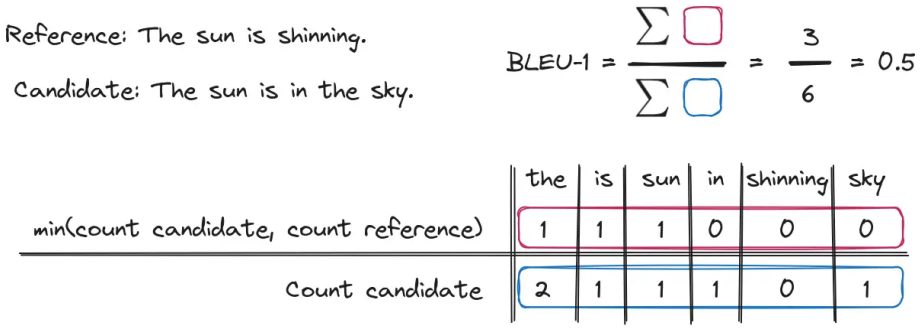


Figure: BLEU score calculation

Outline

1. Introduction
2. Dataset
3. Methodology
4. Metrics
5. Results
6. Discussion & Future Work

Results

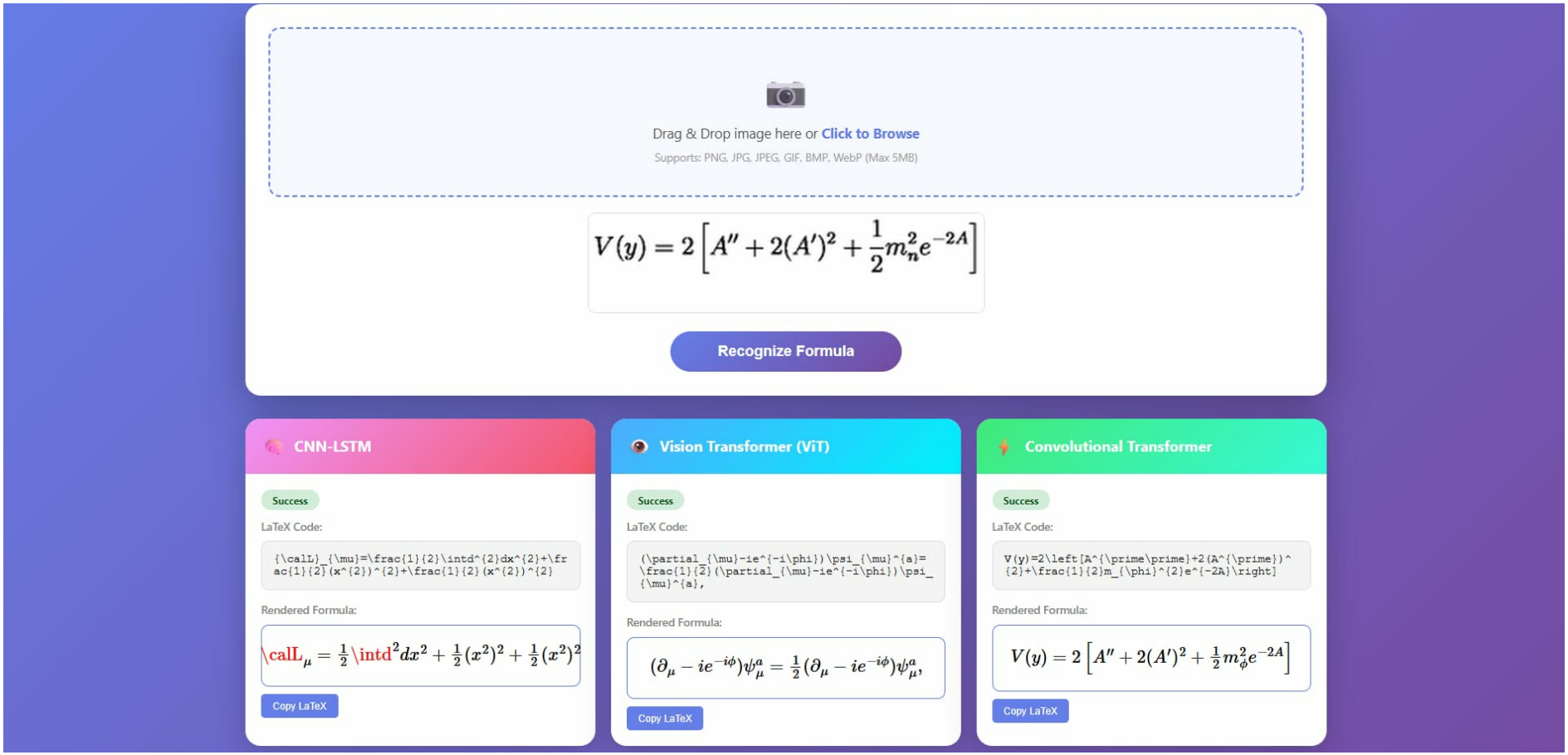
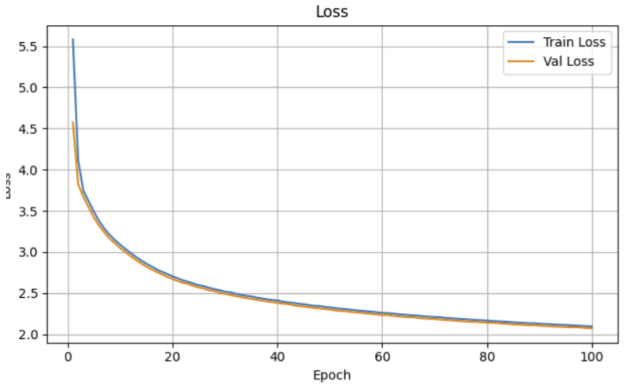
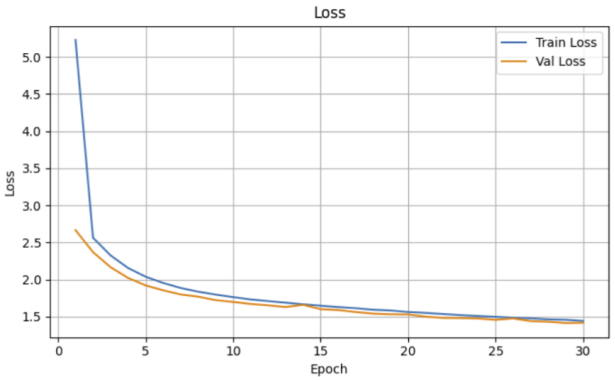


Figure: Demo results of 3 models (CNN-LSTM, ViT, Convolutional Transformer) on the web application.

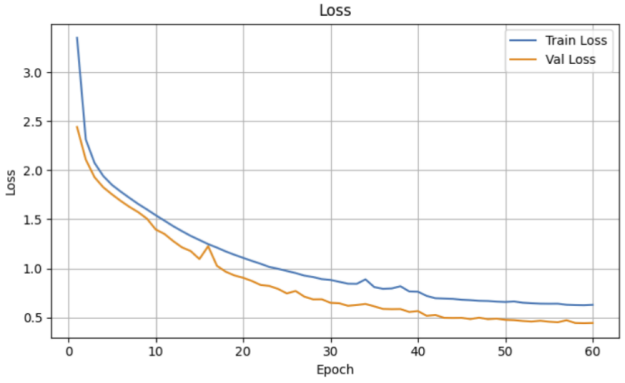
Training Loss



CNN-LSTM



Vision Transformer

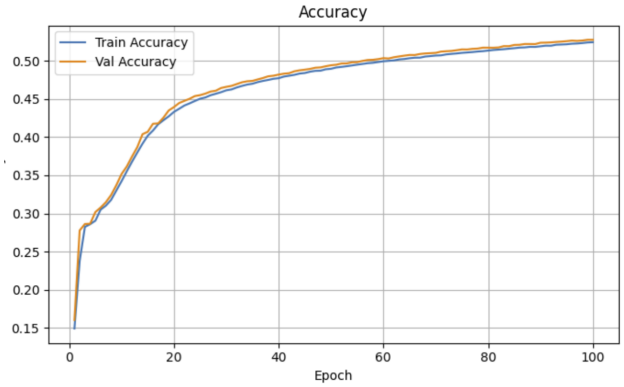


Conv. Transformer

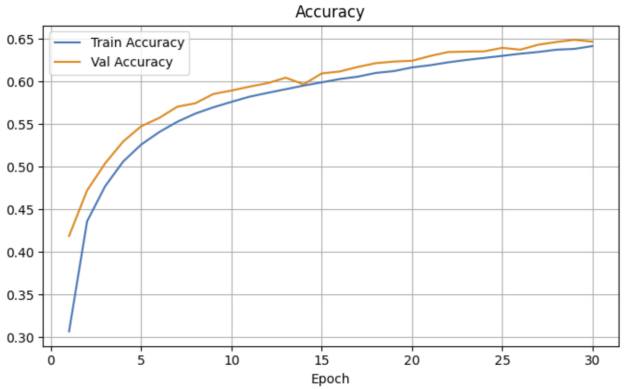
Comments

- CNN-LSTM: Loss of 2.0 after 100 epochs.
- Vision Transformer: Better loss of around 1.5 after 30 epochs.
- Conv. Transformer: Best loss of 0.5 after 60 epochs.
- CNN-LSTM and Vision Transformer seem to be overfitting by the finishing epoch.

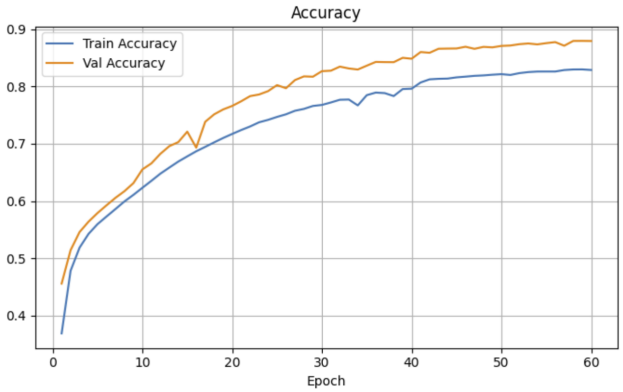
Training Accuracy



CNN-LSTM



Vision Transformer



Conv. Transformer

Comments

- CNN-LSTM: Accuracy of 0.5 after 100 epochs.
- Vision Transformer: Better accuracy of around 0.65 after 30 epochs.
- Conv. Transformer: Best accuracy of 0.9 after 60 epochs.
- CNN-LSTM seems to be overfitting by the finishing epoch.

Test Set Evaluation

Table: Test Set Evaluation

Model	Levenshtein Distance	BLEU Score
CNN-LSTM	0.8765	0.0379
Vision Transformer (ViT)	0.7347	0.0958
Convolutional Transformer	0.3233	0.5405

Comments

- The CNN-LSTM model performed the worst among the three models, showing the highest normalized Levenshtein Distance and the lowest BLEU score.
- The ViT model achieved moderate performance only.
- The Convolutional Transformer model outperformed both CNN-LSTM and ViT by a large distance, achieving the lowest normalized Levenshtein distance and the highest BLEU score.

Outline

1. Introduction
2. Dataset
3. Methodology
4. Metrics
5. Results
6. Discussion & Future Work

Discussion

Learning Points

- The Vocabulary creation task is essential in determining the quality of the code generated by the model.
- Model choice can strongly influence training performance. In this case, a combination of CNN and Transformer outperforms the two simpler architectures.

Points for Improvement

- The performance of ViT is significantly worse than that in the reference paper.
- The training performance can be improved with (a) specified knowledge of the domain and (b) extra steps in training (example: data augmentation and learning rate scheduling)
- The chosen metrics are not totally suitable for the code comparison task.

Future Work

Remarks

- Expand the quality and quantity of the input datasets to include more mathematical symbols to strengthen the generalization of the model.
- Improve training strategies such as data augmentation and learning rate scheduling.
- Explore the problem of Handwritten Mathematical Expression Generation.

Thanks for listening.